



BITS Pilani
K K Birla Goa Campus

INSTRUCTION SET

MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani
Department of CS & IS



Basic Logic Instructions

- AND, OR, Exclusive-OR, and NOT
- Logic operations provide **binary bit control** in low-level software. The logic instructions allow bits to be **set, cleared, or complemented**.
- When binary data are manipulated in a register or a memory location, the rightmost bit position is always numbered bit 0.
- Bit position numbers increase from bit 0 toward the left, to bit 7 for a byte, and to bit 15 for a word.
- A doubleword (32 bits) uses bit position 31 as its leftmost bit and a quadword (64-bits) uses bit position 63 as its leftmost bit.

AND

- The AND operation performs logical multiplication
- 0 AND anything is always 0, and 1 AND 1 is always 1.

A	B	T
0	0	0
0	1	0
1	0	0
1	1	1



AND

Assembly Language

Operation

AND AL,BL	AL = AL and BL
AND CX,DX	CX = CX and DX
AND ECX,EDI	ECX = ECX and EDI
AND RDX,RBP	RDX = RDX and RBP (64-bit mode)
AND CL,33H	CL = CL and 33H
AND DI,4FFFH	DI = DI and 4FFFH
AND ESI,34H	ESI = ESI and 34H
AND RAX,1	RAX = RAX and 1 (64-bit mode)
AND AX,[DI]	The word contents of the data segment memory location addressed by DI are ANDed with AX
AND ARRAY[SI],AL	The byte contents of the data segment memory location addressed by ARRAY plus SI are ANDed with AL
AND [EAX],CL	CL is ANDed with the byte contents of the data segment memory location addressed by ECX

Masking

- AND operation clears bits of a binary number , this task is called masking.

	x x x x	x x x x	Unknown number
•	0 0 0 0	1 1 1 1	Mask
	0 0 0 0	x x x x	Result

- The OR instruction uses any of the addressing modes allowed to any other instruction except segment register addressing.

Examples

```
AX=0022 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0103 NV UP EI PL NZ NA PE NC
0859:0103 BB1100          MOV     BX,0011
```

```
AX=0022 BX=0011 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0106 NV UP EI PL NZ NA PE NC
0859:0106 21D8          AND     AX,BX
```

```
AX=0000 BX=0011 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0108 NV UP EI PL ZR NA PE NC
0859:0108 3000          XOR     [BX+SI],AL
```

Masking Example

- ASCII-coded number can be converted to BCD using masking
- ASCII 30H to 39H -> 0 to 9

```
MOV BX, 3135H      ;load ASCII  
AND BX, 0F0FH      ;mask BX
```

Masking Example

```
MOV BX,3135H           ;load ASCII
AND BX,0F0FH           ;mask BX
```

```
AX=3135 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0103 NU UP EI NG NZ NA PE NC
0859:0103 BB0F0F          MOV     BX,0F0F
```

```
AX=3135 BX=0F0F CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0106 NU UP EI NG NZ NA PE NC
0859:0106 21D8          AND     AX,BX
```

```
AX=0105 BX=0F0F CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0108 NU UP EI PL NZ NA PE NC
0859:0108 0000          ADD     [BX+SI],AL
```


OR Gate

- Performs logical addition and is often called the Inclusive-OR function.

A	B	T
0	0	0
0	1	1
1	0	1
1	1	1



Masking with OR

$$\begin{array}{r} \text{x x x x x x x x} \quad \text{Unknown number} \\ + \quad 0 0 0 0 1 1 1 1 \quad \text{Mask} \\ \hline \text{x x x x 1 1 1 1} \quad \text{Result} \end{array}$$

OR Examples

<i>Assembly Language</i>	<i>Operation</i>
OR AH,BL	AL = AL or BL
OR SI,DX	SI = SI or DX
OR EAX,EBX	EAX = EAX or EBX
OR R9,R10	R9 = R9 or R10 (64-bit mode)
OR DH,0A3H	DH = DH or 0A3H
OR SP,990DH	SP = SP or 990DH
OR EBP,10	EBP = EBP or 10
OR RBP,1000H	RBP = RBP or 1000H (64-bit mode)
OR DX,[BX]	DX is ORed with the word contents of data segment memory location addressed by BX
OR DATES[DI + 2],AL	The byte contents of the data segment memory location addressed by DI plus 2 are ORed with AL

Example

If we multiply two BCD numbers (5,7) , can you convert its result to ASCII?

Example

If we multiply two BCD numbers (5,7) , can you convert its result to ASCII?

Example

If we multiply two BCD numbers (5,7) , can you convert its result to ASCII?

```
tempAL := AL;
```

```
AH := tempAL / imm8; (* imm8 is set to 0AH for the AAM mnemonic *)
```

```
AL := tempAL MOD imm8;
```

AAM →

Example

If we multiply two BCD numbers (5,7) , can you convert its result to ASCII?

AAM → `tempAL := AL;`
`AH := tempAL / imm8; (* imm8 is set to 0AH for the AAM mnemonic *)`
`AL := tempAL MOD imm8;`

```
MOV    AL, 5                ;load data
MOV    BL, 7
MUL    BL
AAM                      ;adjust
OR     AX, 3030H           ;convert to ASCII
```

Exclusive-OR

- The Exclusive-OR operation excludes this condition; the Inclusive-OR includes it.

A	B	T
0	0	0
0	1	1
1	0	1
1	1	0



- The XOR instruction uses any addressing mode except segment register addressing.

XOR

- The Exclusive-OR instruction is useful if some bits of a register or memory location must be inverted.
- This instruction allows part of a number to be inverted or complemented.

$$\begin{array}{rcl}
 x & x & x & x & x & x & x & x & \text{Unknown number} \\
 \oplus & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & \text{Mask} \\
 \hline
 x & x & x & x & \bar{x} & \bar{x} & \bar{x} & \bar{x} & \text{Result}
 \end{array}$$

XOR

- Use for the Exclusive-OR instruction is to clear a register to zero.
- E.g. XOR CH,CH
- instruction clears register CH to 00H and requires 2 bytes of memory to store the instruction.
- E.g. MOV CH, 00H instruction also clears CH to 00H, but requires 3 bytes of memory.
- Because of this saving, the XOR instruction is often used to clear a register in place of a move immediate

So far..

- The AND instruction clears (0) bits
 - The OR instruction sets (1) bits
 - Exclusive-OR instruction inverts bits.
-
- This is ideal for control system applications in which equipment must be turned on (1), turned off (0), and toggled from on to off or off to on.

Test and Bit Test Instructions

- The TEST instruction performs the **AND operation**.
- The difference is that the AND instruction **changes the destination operand**, whereas the TEST instruction does not.
- A TEST only **affects the condition of the flag register**, which indicates the result of the test.
- The TEST instruction uses the same addressing modes as the AND instruction.
- The TEST instruction functions in the same manner as a CMP instruction.
- The difference is that the TEST instruction normally **tests a single bit** (or occasionally multiple bits), whereas the CMP instruction **tests the entire byte, word, or doubleword**.

TEST Example

- TEST instruction is followed by either the JZ (jump if zero) or JNZ (jump if not zero) instruction.
- The destination operand is normally tested against immediate data.
- The value of immediate data is 1 to test the rightmost bit position, 2 to test the next bit, 4 for the next, and so on

```
TEST AL,1           ;test right bit
JNZ  RIGHT          ;if set
TEST AL,128         ;test left bit
JNZ  LEFT           ;if set
```

TEST

- It only updates the flags based on the outcome of the AND operation. The affected flags include:
- **Zero Flag (ZF):** Set if the result of the AND operation is zero.
- **Parity Flag (PF):** Set if the number of set bits in the result is even.
- **Overflow Flag (OF):** Cleared (reset) to zero.

Examples

<i>Assembly Language</i>	<i>Operation</i>
TEST DL,DH	DL is ANDed with DH
TEST CX,BX	CX is ANDed with BX
TEST EDX,ECX	EDX is ANDed with ECX
TEST RDX,R15	RDX is ANDed with R15 (64-bit mode)
TEST AH,4	AH is ANDed with 4
TEST EAX,256	EAX is ANDed with 256

NOT & NEG

- Logical inversion, or the one's complement (NOT), and arithmetic sign inversion, or the two's complement (NEG)
- These are two of a few instructions that contain only one operand.
- NOT and NEG can use any addressing mode except segment register addressing

NOT CH

CH is one's complemented

NEG CH

CH is two's complemented

NEG AX

AX is two's complemented

NOT EBX

EBX is one's complemented

NEG ECX

ECX is two's complemented

NOT RAX

RAX is one's complemented (64-bit mode)

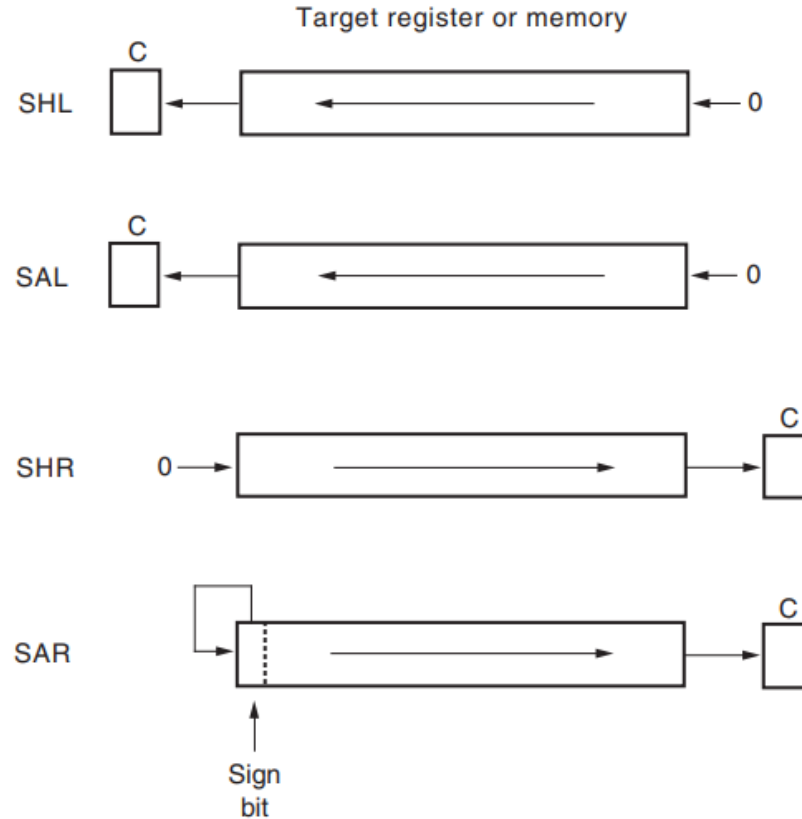
SHIFT AND ROTATE

- Shift and rotate instructions manipulate binary numbers at the **binary bit level**, as did the AND, OR, Exclusive-OR, and NOT instructions.
- Shifts and rotates find their most common applications in low-level software used to control I/O devices.

Shift

- Shift instructions position or move numbers to the left or right within a register or memory location.
- They also perform simple arithmetic such as multiplication by powers of (left shift) and division by powers of 2^{-n} (right shift).
- The microprocessor's instruction set contains four different shift instructions:
 - Two are logical shifts and two are arithmetic shifts

Shift



Shift Instruction

- Logical shift operations function with unsigned numbers, and arithmetic shifts function with signed numbers.
- Logical shifts multiply or divide unsigned data, and arithmetic shifts multiply or divide signed data.
- A shift left always multiplies by 2 for each bit position shifted, and a shift right always divides by 2 for each bit position shifted.
- Shifting a number two places, to the left or right, multiplies or divides by 4.

Shift Instruction

- Shift the DX register left 14 places in two different ways.

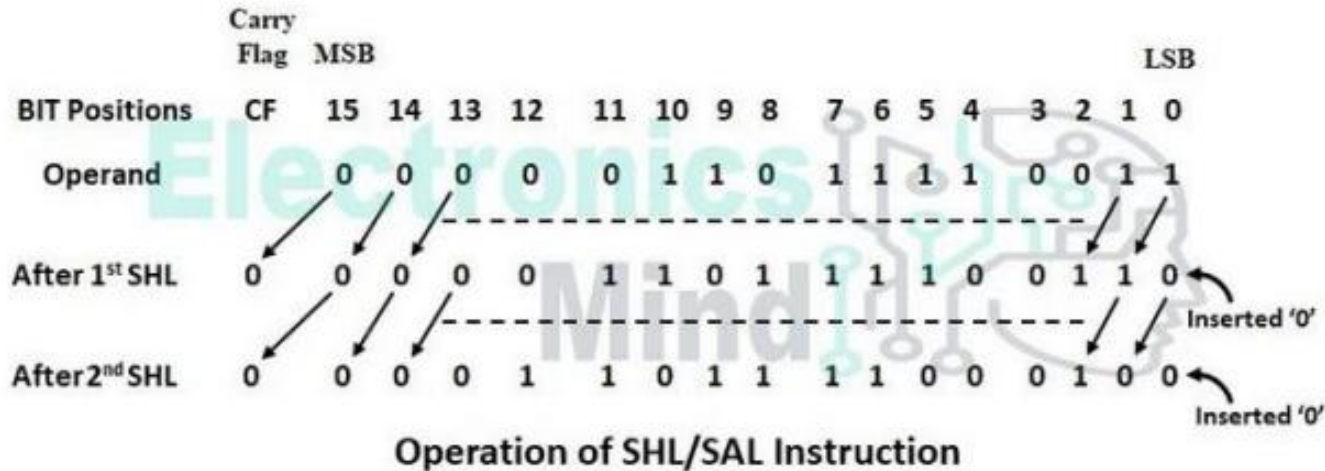
```
SHL DX, 14
```

OR

```
MOV CL, 14  
SHL DX, CL
```

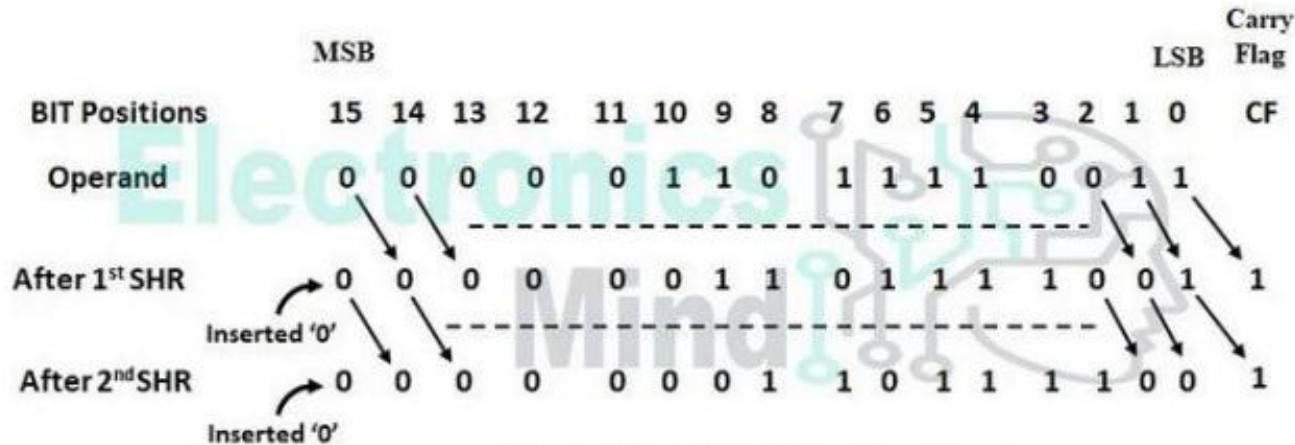
SHL/SAL Instruction

- SHL/SAL instruction performs shifting of each bit in the operand (register or memory location) to the left side and fills the least significant bit (LSB) with zero and obtain the result in the destination operand.
- The MSB is shifted into the carry flag (CF).



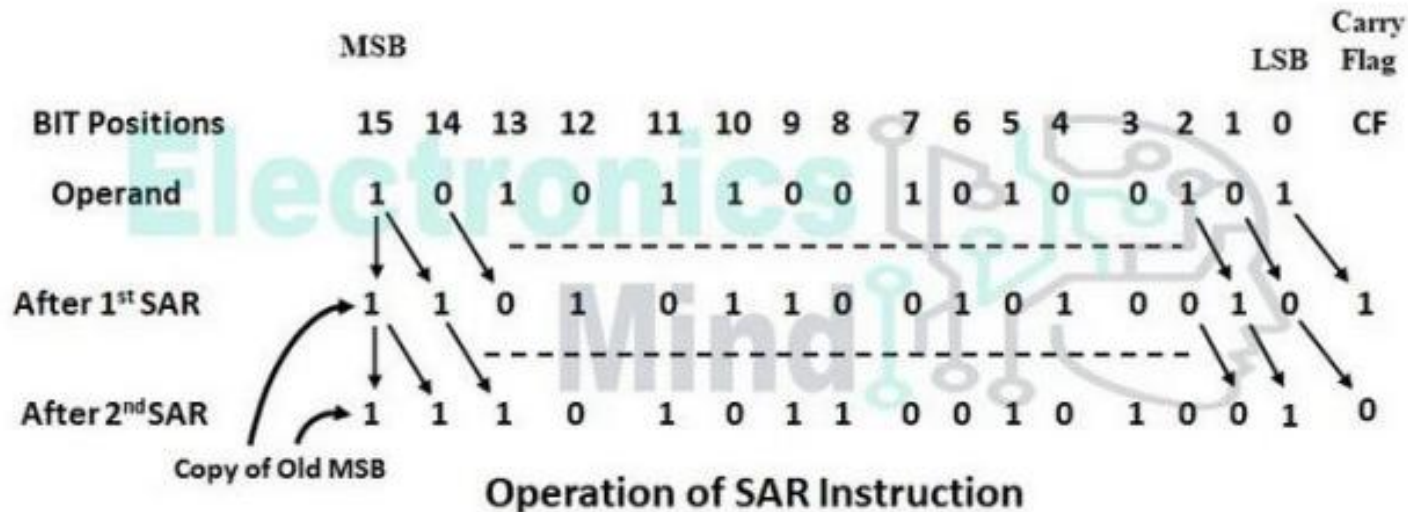
SHR Instruction

- The SHR instruction performs shifting of each bit in the operand (register or memory location) to the right side and fills the most significant bit (MSB) with zero.
- The LSB is shifted into the carry flag (CF).



SAR Instruction

- It does the same operation as SHR except it fills the bit portion shifted right from the MSB with a copy of old MSB i.e., the SAR instruction performs the right shift on each bit and fills the MSB portion with the copy of old MSB.



Rotate

- Rotate instructions position binary data by rotating the information in a register or memory location, either from one end to another or through the carry flag.
- Numbers rotate through a register or memory location, through the C flag (carry), or through a register or memory location only.
- With either type of rotate instruction, the programmer can select either a left or a right rotate.
- A rotate count can be immediate or located in register CL.
- If CL is used for a rotate count, it does not change.

Example

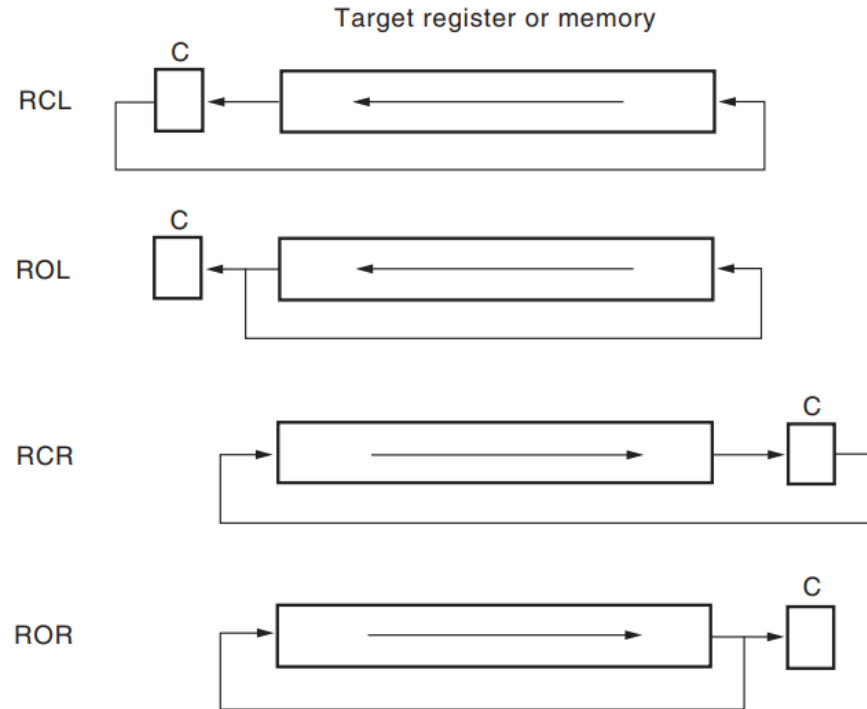
- Shifts the 48-bit number in registers DX, BX, and AX left one binary place.

```
SHL  AX, 1
```

```
RCL  BX, 1
```

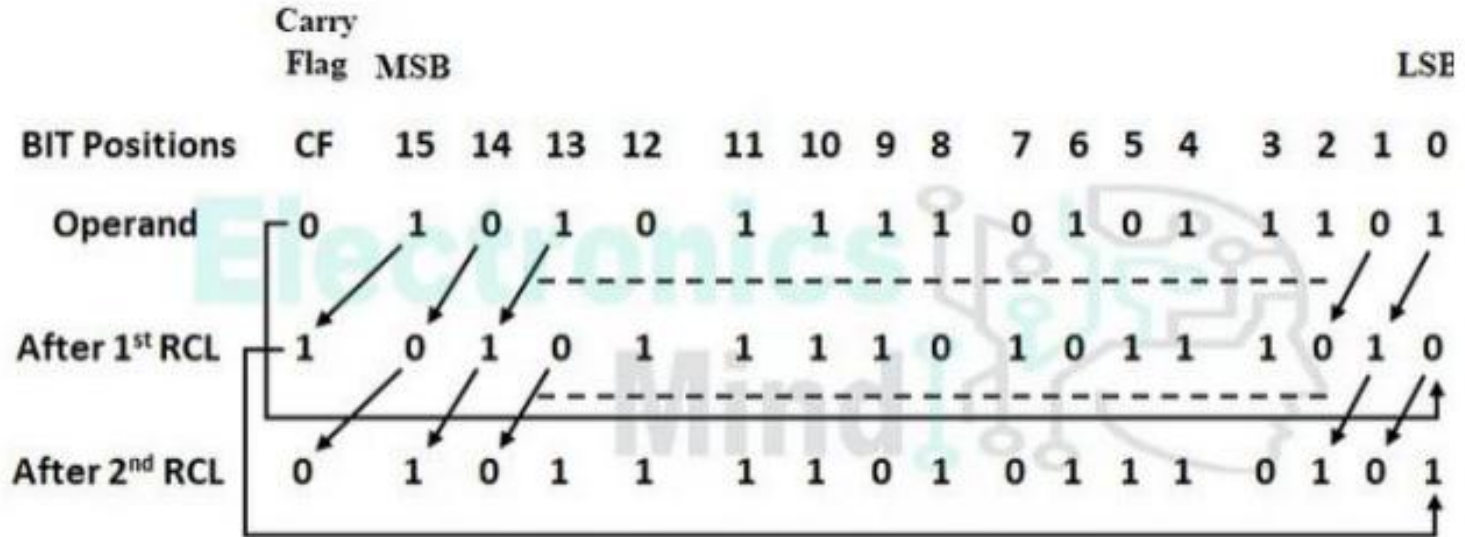
```
RCL  DX, 1
```

Rotate Instruction



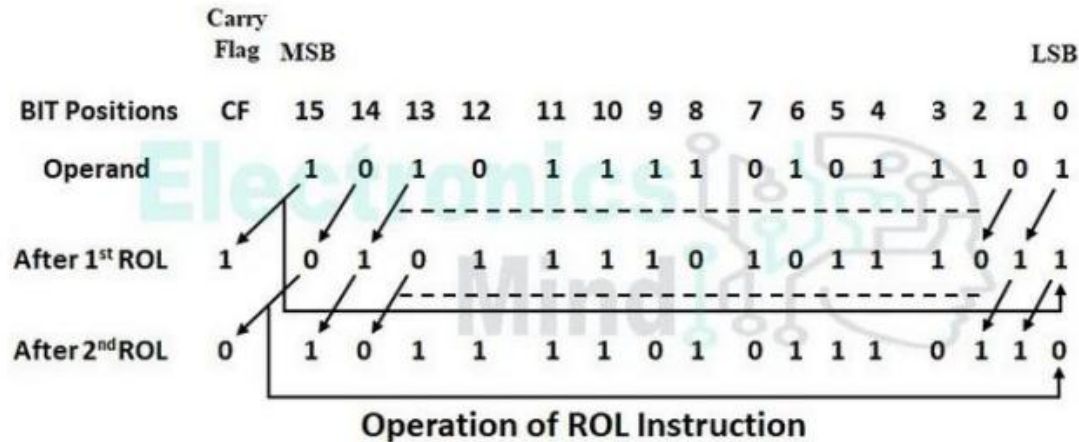
RCL Instruction

- The contents of the operand will be rotated bit-wise left with the carry flag involved in the rotation.



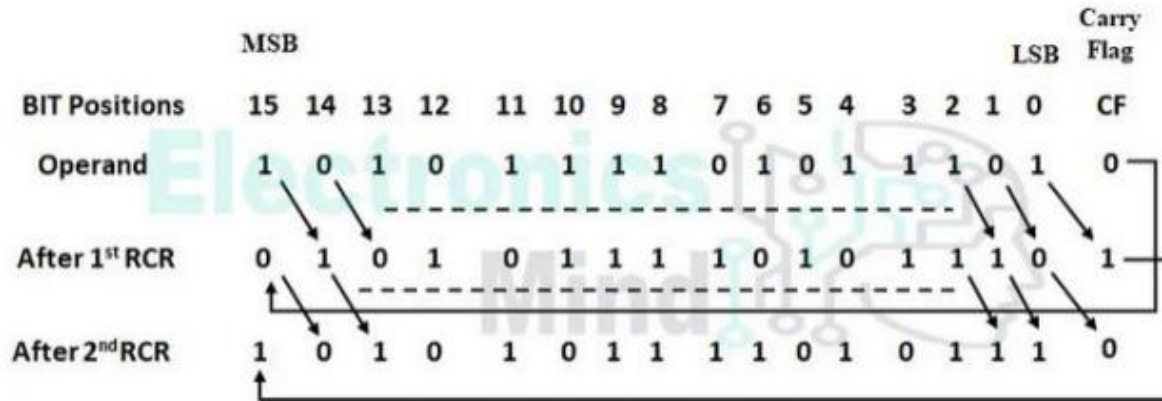
ROL Instruction

- The contents of the operand (register or memory location) are rotated left bit-wise by some number of positions depending on the count value.
- During this rotation, the most significant bit (MSB) is moved into the carry flag (CF) as well as into the least significant bit (LSB) position.



RCR Instruction

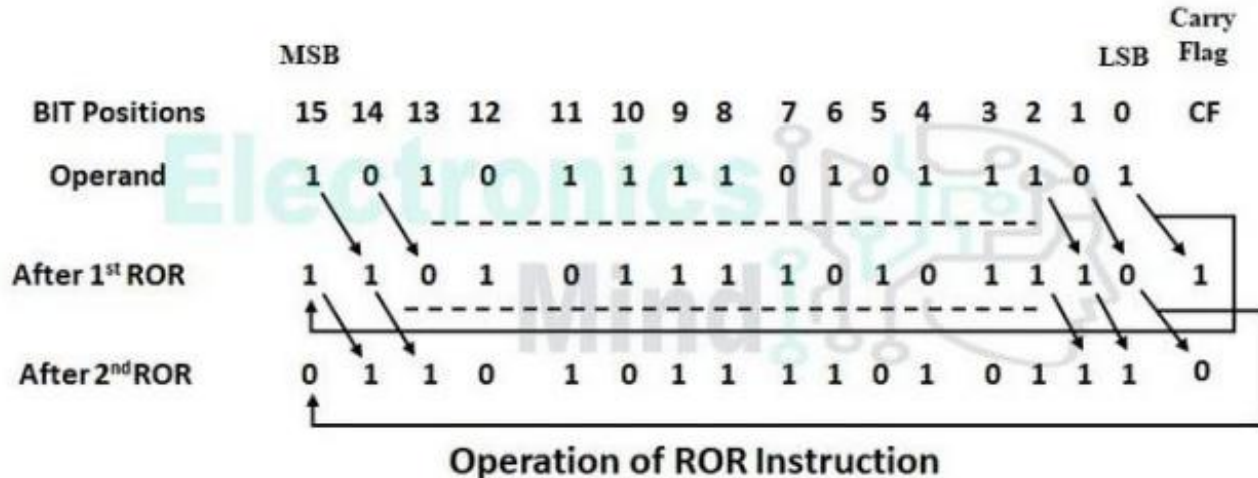
- In this instruction, the contents of the operand (register or memory location) are rotated right bit-wise by some number of positions along with the carry flag.
- During the rotation the least significant bit (LSB) is moved to carry flag, and the carry flag is moved into the most significant bit (MSB).



Operation of RCR Instruction

ROR Instruction

- The contents of the operand are rotated right bit-wise by some number of positions depending on the count value.
- The ROL instruction seen above rotates the bits towards the left side, whereas the ROR instruction rotates the bits towards the right side.
- Since this instruction rotates the bits right, the least significant bit (LSB) is moved into the carry flag (CF) as well as into the most significant bit (MSB) position.





BITS Pilani
K K Birla Goa Campus

THANK YOU





Decimal Numbers	4-bit Binary Number	Hexadecimal Number
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F



BITS Pilani
K K Birla Goa Campus

